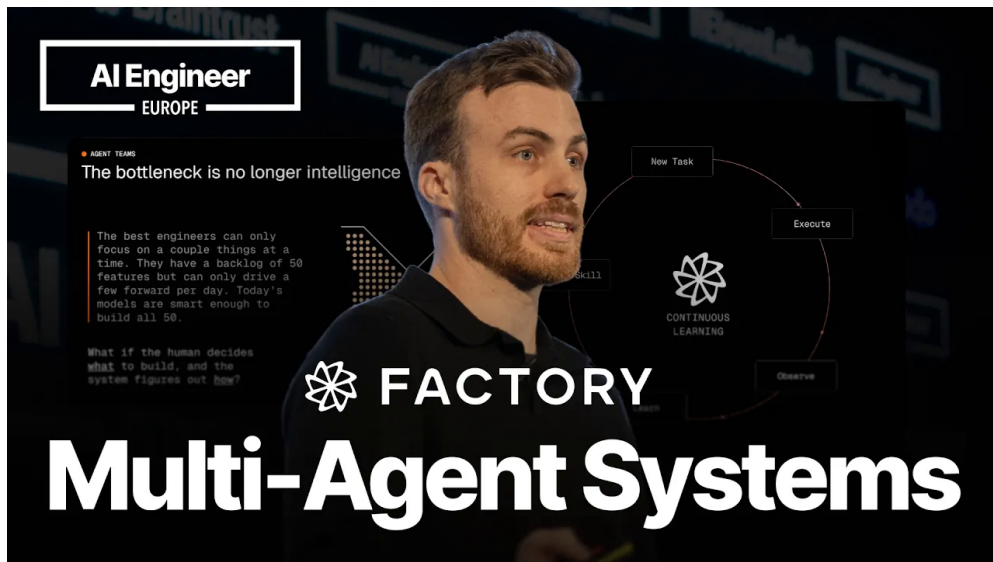


课程笔记

The Multi-Agent Architecture That Actually Ships

五道口纳什 & Codex

2026 年 5 月 11 日



视频作者/频道: AI Engineer

发布日期: 2026-05-06

视频时长: 18:30

视频链接: <https://www.youtube.com/watch?v=ow1we5PzK-o>

目录

1 引言：软件工程的瓶颈正在转移	3
1.1 演讲者背景与 Factory 的使命	3
1.2 核心论断：瓶颈是人类注意力	3
1.3 本章小结	4
2 五种前沿多智能体框架分类	5
2.1 委托 (Delegation)	5
2.2 创造者-验证者 (Creator-Verifier)	5
2.3 直接通信 (Direct Communication)	6
2.4 协商 (Negotiation)	6
2.5 广播 (Broadcast)	6
2.6 本章小结	7
3 Missions 核心架构：三角色与验证体系	7
3.1 Missions 概述与三角色架构	7
3.1.1 编排器：规划者与需求澄清者	7
3.1.2 工作者：干净上下文的实现者	8
3.1.3 验证器：独立视角的检验者	8
3.2 验证契约 (Validation Contract)	9
3.2.1 先写代码后补测试的陷阱	9
3.2.2 验证契约的定义与结构	10
3.2.3 两类验证器：审查验证器与用户测试验证器	10
3.2.4 对抗性验证的设计	10
3.3 结构化交接 (Structured Handoff)	11
3.3.1 超越”我完成了”	11
3.3.2 里程碑边界的自我纠正	11
3.3.3 支撑长期运行的结构	12
3.4 本章小结	12
4 执行策略与监控	14
4.1 串行执行与内部并行化	14
4.1.1 为何并行执行在软件开发中失效	14
4.1.2 串行执行：全局一致性优先	15
4.1.3 内部并行化：只读操作的安全加速	15
4.2 Mission Control：监控与异步运行	15
4.2.1 超越聊天界面的监控需求	15
4.2.2 Mission Control 的核心功能	16
4.2.3 两种监督模式	16
4.3 本章小结	17

5	模型选择、生产数据与架构未来	18
5.1	按角色选择模型: Droid Whispering	18
5.2	生产数据分析: 构建 Slack 克隆	19
5.3	企业应用与 Bitter Lesson	20
5.4	本章小结	21
6	总结与延伸	22
6.1	演讲者的核心主张回顾	22
6.2	核心主张的结构化提炼	23
6.3	跨章节综合与概念压缩	23
6.4	具体收获	24
6.5	开放问题与后续步骤	25
6.6	结语	25

1 引言：软件工程的瓶颈正在转移

1.1 演讲者背景与 Factory 的使命

演讲者 Luke Alvoeiro 在开场提出了一个雄心勃勃的目标：在二十分钟的演讲结束后，听众应当能够组建出可以完成比单智能体（single agent）难一个数量级的任务的多智能体团队。Luke 拥有深厚的开发者工具（dev tools）背景——约两年半前，他在 Block 公司启动了一个后来演变为 Goose 的项目。Goose 如今已成为领先的开源编程智能体之一，并已被捐赠给 Agentic AI Foundation。

目前，Luke 在 Factory 领导核心智能体系统（core agent harness）的研发。Factory 的使命是将自主性（autonomy）带入整个软件开发生命周期（software development life cycle），而不仅仅是代码生成这一孤立环节。

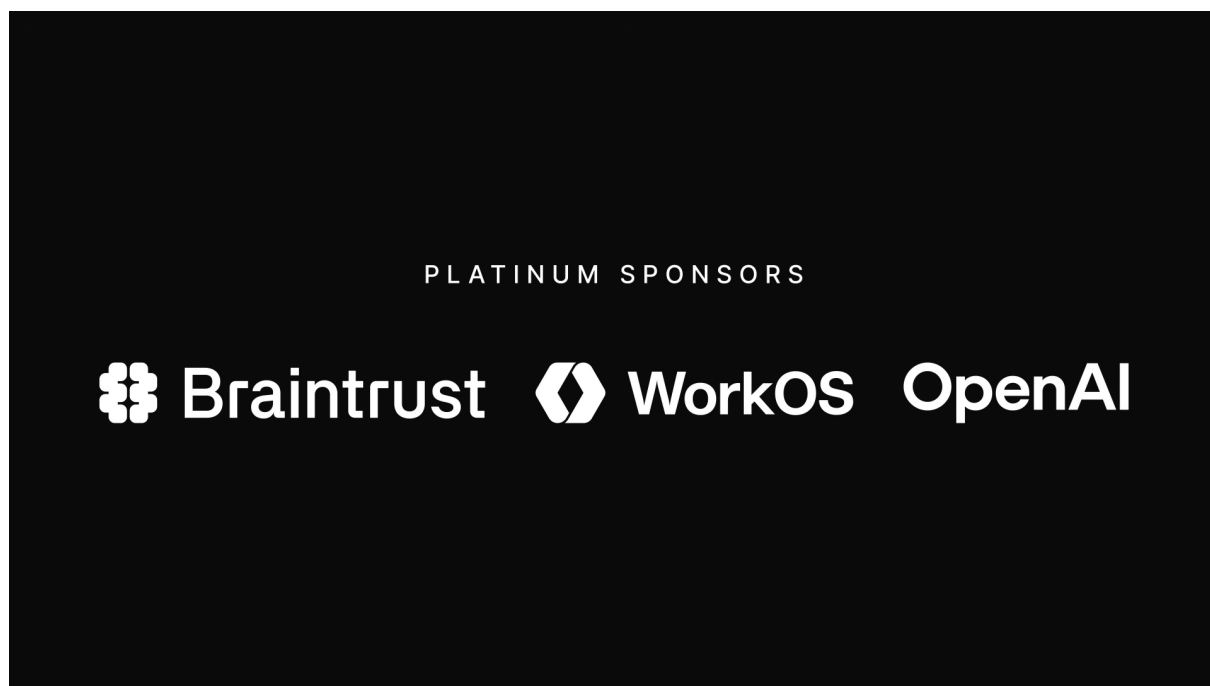


图 1: 演讲者 Luke Alvoeiro 自我介绍，回顾从 Block 的 Goose 项目到 Factory 的经历。¹

背景知识：Goose 与开源编程智能体

Goose 是 Block 公司开源的编程智能体项目，属于当前编码智能体（coding agent）领域的重要代表之一。它能够在开发者环境中自主执行代码编辑、测试运行、命令执行等任务。Goose 被捐赠给 Agentic AI Foundation 一事，标志着编程智能体正从单一公司项目向社区化、标准化方向演进。

1.2 核心论断：瓶颈是人类注意力

Luke 提出了本次演讲的核心论断：

¹ 视频画面时间区间：00:00:15–00:00:20。

核心论断

当今软件工程的瓶颈不是智能 (intelligence)，而是人类注意力 (human attention) 的有限性。

他以一个具体场景说明这一论断：即便是最优秀的工程师，每天也只能推进少量任务。他们的待办清单上可能有 50 个功能需求 (features)，但每天只能驱动其中几个向前推进——因为每个任务都需要关注，每次提交都需要审查。今天的模型已经足够聪明，能够理解并完成这 50 个任务中的任何一个，但人类没有足够的监督带宽 (supervision bandwidth) 来逐一审阅其实现过程。

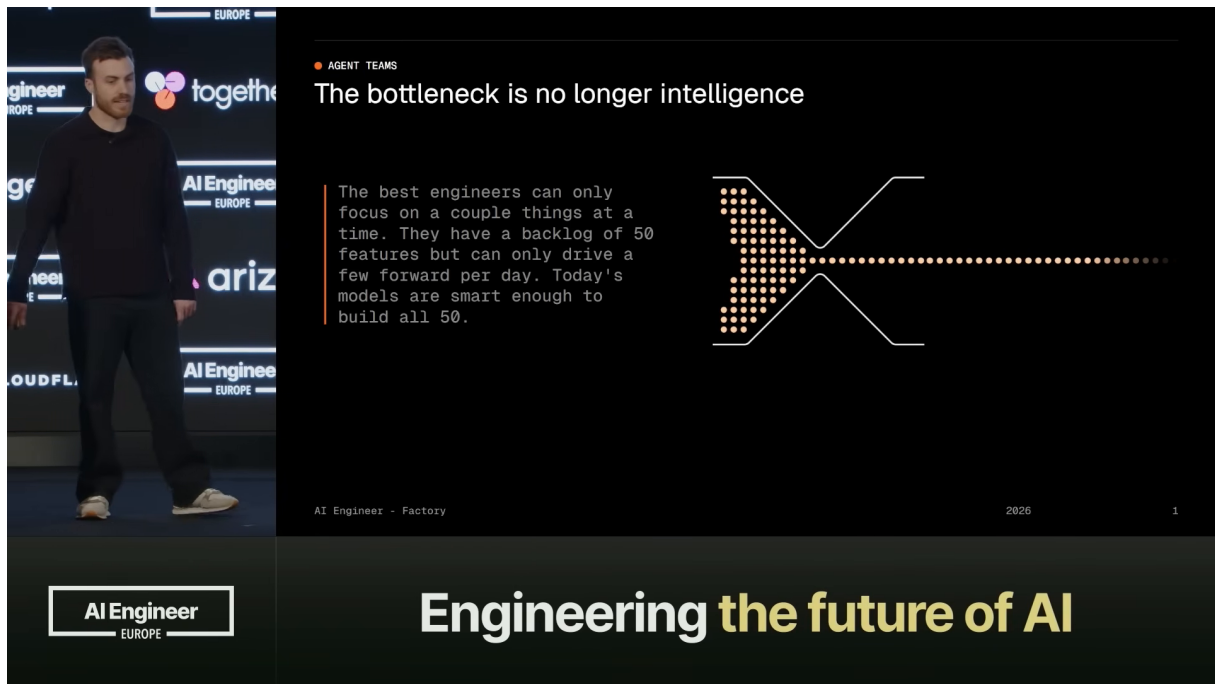


图 2: 核心论点：软件工程的瓶颈已从”智能不足”转变为”人类注意力有限”。²

由此引出 Factory 的核心问题：如果由人类决定”做什么”，而由系统自主决定”怎么做”，将会发生什么？一个智能体可以连续工作数小时甚至数天，而人类只需在完成后回来验收成果。这正是 Missions 系统试图回答的问题，也是后续章节将要深入探讨的架构基础。

常见误解

许多人认为多智能体系统的价值在于”更多智能体 = 更多智能”。Luke 的论断直接挑战了这一直觉：真正的瓶颈不在于模型是否足够聪明，而在于人类是否有足够的时间和精力去监督它们。因此，多智能体架构的设计目标应当是减少人类监督负担，而非单纯堆砌智能体数量。

1.3 本章小结

本章从演讲者的背景出发，引出了软件工程领域一个根本性的认知转变：随着大语言模型能力的快速提升，限制因素已从”智能不足”转向”人类注意力有限”。这一论断为后续所有架构设计提供了动机基础——Missions 系统的每一个设计选择，本质上都是在回答”如何在人类监督带宽有限的前提下，让智能体系统可靠地运行数天”。

²视频画面时间区间：00:01:10–00:01:14。

2 五种前沿多智能体框架分类

在介绍 Missions 的具体架构之前，Luke 首先对当前多智能体系统领域进行了梳理。他指出，这一领域目前略显混乱——每个框架都有自己的术语、自己的方法论、自己对“什么有效”的见解。为此，他提出了一个简化的分类体系（taxonomy），将前沿多智能体框架归纳为五种基本通信模式。

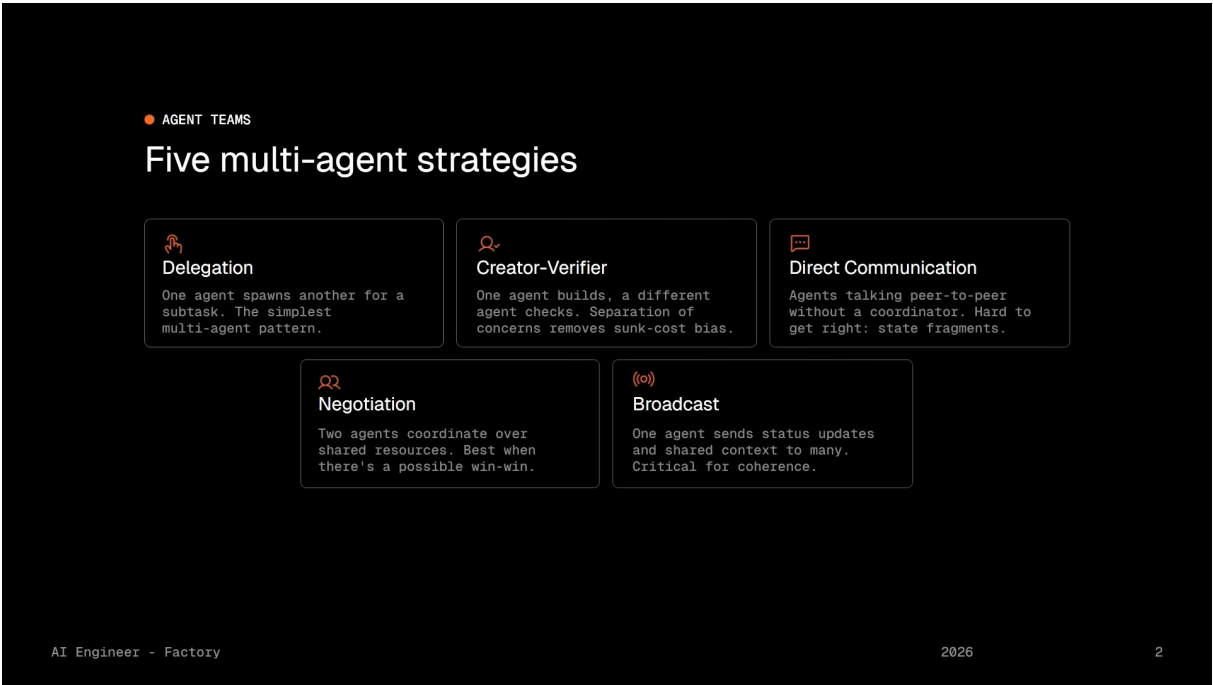


图 3: 五种前沿多智能体框架分类：委托、创造者-验证者、直接通信、协商、广播。³

2.1 委托 (Delegation)

定义：委托 (Delegation)

一个智能体生成子智能体（sub-agent）并为其分配任务，父智能体等待子智能体返回结果后继续进行自身工作。

委托是最简单的多智能体通信形式，也是大多数人首先实现的模式。典型场景是编码工具中的子智能体：父智能体可能说“去弄清楚数据库结构”，子智能体完成研究后返回结果，父智能体基于该结果继续执行。这种模式的结构清晰、易于理解，但父智能体在子智能体运行期间处于阻塞等待状态。

2.2 创造者-验证者 (Creator-Verifier)

定义：创造者-验证者 (Creator-Verifier)

一个智能体负责构建（build），另一个独立的智能体负责检查（check）前者的工作成果。关键在于职责分离（separation of concerns）与消除确认偏误（confirmation bias）。

³视频画面时间区间：00:02:07–00:02:12。

创造者-验证者模式的核心洞察来自认知心理学：实现代码的智能体存在确认偏误——它希望代码能工作。一个拥有全新上下文的独立智能体，远比原实现者更容易发现问题。这正是人类工程实践中代码审查（code review）机制的原理所在。该模式的价值不在于增加一个检查步骤，而在于引入一个与实现者利益无关的独立视角。

2.3 直接通信 (Direct Communication)

定义：直接通信 (Direct Communication)

智能体之间无需中央协调器（central coordinator），直接进行点对点通信，类似于“私信”（DMing each other）。

直接通信模式在理论上具有去中心化的优雅性，但在实践中难以正确实现。根本难点在于状态碎片化（state fragmentation）：当多个智能体在没有单一事实来源（single source of truth）的情况下自由对话时，每个智能体持有的局部状态可能相互矛盾，导致系统缺乏全局一致性。这一模式对长期运行任务尤其危险，因为状态漂移会随时间累积。

设计陷阱：直接通信的状态碎片化

直接通信常被误认为“更灵活”或“更接近自然协作”。然而，在软件工程场景中，缺乏中央协调器意味着没有统一的上下文视图。当两个智能体对同一段代码持有不同理解时，冲突将在后续阶段以难以调试的方式爆发。

2.4 协商 (Negotiation)

定义：协商 (Negotiation)

智能体围绕共享资源（shared resource）进行通信，例如争夺同一 API 的使用权或同一代码区域的修改权。

协商模式的关键在于：它不必是对抗性的（adversarial）。事实上，最佳应用场景是存在正和博弈（positive sum trading）的情况——即智能体之间存在双赢（win-win）交互的可能。例如，两个智能体都需要修改数据库层，但通过协商可以协调修改顺序，避免冲突，最终双方都获益。这与经济学中的比较优势原理类似：通过协调而非竞争，整体产出得以提升。

2.5 广播 (Broadcast)

定义：广播 (Broadcast)

一个智能体向多个智能体同时发送信息，包括状态更新、适用于所有人的新上下文、共享约束（shared constraints）等。

广播是五种模式中最“不炫目”的一种，但对于长期任务的一致性维护至关重要。当系统运行数小时甚至数天时，每个智能体都需要及时获知全局状态的变更——新的约束条件、已完成的里程碑、发现的问题等。广播机制确保了信息能够高效地触达所有相关智能体，而不需要逐一建立点对点连接。

设计权衡：五种模式的互补性

这五种模式并非互斥的替代方案，而是可以组合使用的构建块（building blocks）。Luke 特别指出，Missions 系统后续将委托、创造者-验证者、广播和协商四种模式组合为单一工作流——直接通信被有意排除，正是因为其状态碎片化的风险与长期运行任务的需求相冲突。

2.6 本章小结

本章建立了多智能体通信模式的系统化认知框架。五种模式各有其适用场景与内在风险：委托简单直观但父智能体阻塞；创造者-验证者通过职责分离提升质量；直接通信灵活但面临状态碎片化难题；协商在共享资源场景下实现协调；广播为长期一致性提供信息基础设施。理解这些模式的特性与局限，是后续深入 Missions 架构设计的必要铺垫。Missions 的核心创新之一，正在于有意识地将其中四种模式组合起来，同时规避直接通信的陷阱，从而支撑起能够连续运行数天的自主软件工程工作流。

3 Missions 核心架构：三角色与验证体系

从五种框架到统一工作流

在上一节中，我们介绍了五种多智能体通信框架：**委托**（Delegation）、**创造者-验证者**（Creator-Verifier）、**直接通信**（Direct Communication）、**协商**（Negotiation）和**广播**（Broadcast）。Missions 的独到之处在于，它并非发明第六种框架，而是将前四种框架——委托、创造者-验证者、广播和协商——组合为单一工作流。直接通信之所以被排除，正是因为它缺乏中央协调器，导致状态碎片化，无法支撑长期运行任务的一致性维护。

3.1 Missions 概述与三角色架构

当我们掌握了五种基础通信模式后，一个自然的问题浮现出来：如何将这些积木组装成一个能够连续运行数天、甚至数周的系统？

Factory 给出的答案是 **Missions**（任务系统）。Luke 明确指出，Missions 不是一个单智能体会话，而是一个**智能体生态系统**（Agent Ecosystem）：多个智能体通过**结构化交接**（Structured Handoff）和**共享状态**（Shared State）进行协作。这一生态系统的核心是三角色架构。

3.1.1 编排器：规划者与需求澄清者

编排器（Orchestrator）是三角色中的”大脑”。当用户描述目标后，编排器扮演的角色类似于一位经验丰富的技术合伙人——它会提出战略性问题，检查需求空间中是否存在模糊不清的地方，然后产出一份完整的计划。

这份计划包含三个核心要素：

- **功能列表**（Features）：将大目标拆解为可独立实现的功能单元；
- **里程碑**（Milestones）：定义阶段性检查点，验证在里程碑边界运行；
- **验证契约**（Validation Contract）：在编码开始之前就定义”完成”的标准——这是 Missions 区别于几乎所有其他系统的关键设计。

⁴图 4 截取自演讲视频 00:04:38 处。

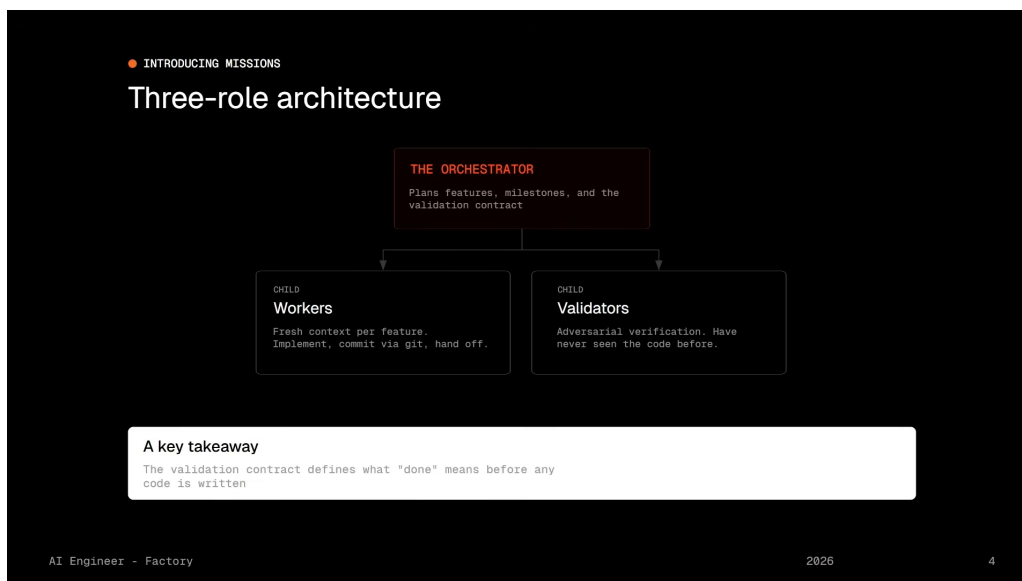


图 4: Missions 三角色架构：编排器（Orchestrator）负责规划与验证契约，工作者（Workers）负责实现，验证器（Validators）负责验证。三者通过共享状态和结构化交接协同工作。⁴

Luke 特别强调，验证契约之所以在规划阶段编写，是因为它必须**独立于实现**来定义正确性。这一点我们将在下一小节深入展开。

3.1.2 工作者：干净上下文的实现者

工作者（Workers）负责具体的功能实现。当一个功能被分配给工作者时，该工作者拥有**干净的上下文（Clean Context）**：没有累积的对话包袱，没有衰减的注意力。

工作者的执行流程极为简洁：

1. 读取编排器为其定义的功能规格说明（Spec）；
2. 实现该功能；
3. 提交代码（Git Commit），为下一个工作者留下干净的基线和工作代码库。

这种设计暗含一个深刻的工程直觉：人类工程师在长时间编码后会出现注意力衰减和上下文污染，智能体亦然。通过让每个工作者只负责一个功能，并在完成后立即交接，Missions 避免了“一个智能体会话越跑越偏”的经典陷阱。

3.1.3 验证器：独立视角的检验者

验证器（Validators）负责验证实现是否符合契约。大多数系统仅运行 lint、类型检查和单元测试，或许再加上代码审查。Missions 不仅做这些，还额外引入了**行为验证（Behavioral Validation）**——不只问“代码看起来对吗？”，而是问“端到端地，这个功能真的工作吗？”

正是这一差异，使得 Missions 能够连续运行数小时、数天而不发生**漂移（Drift）**。Luke 坦言，让这一点成立需要“彻底重新思考验证”——这正是下一小节的核心内容。

三角色架构的设计哲学

Missions 的三角色架构本质上是对软件工程最佳实践的形式化：

- 编排器 $\approx$ 产品经理 + 架构师，负责”做什么”和”做到什么程度算完成”；
- 工作者 $\approx$ 开发工程师，负责”怎么做”，拥有干净的实现环境；
- 验证器 $\approx$ QA 工程师 + 代码审查者，拥有独立视角，从未见过实现代码。

三者之间通过共享状态和结构化交接耦合，而非通过直接通信——这避免了状态碎片化，确保了单一事实来源。

3.2 验证契约 (Validation Contract)

验证契约是 Missions 架构中最具原创性的设计之一。要理解它的必要性，我们必须先审视一个普遍存在的反模式。

3.2.1 先写代码后补测试的陷阱

如果你使用过编码智能体，很可能见过这样的模式：智能体构建了一个功能，写了一些测试，测试全部通过，覆盖率 100%——但功能本身却是错误的。

问题出在哪里？Luke 用一句话点破要害：

”Tests written after implementation don’t catch bugs. They confirm decisions.”

测试是在实现之后编写的，因此它们的形状被代码所塑造，而非被代码试图达成的目标所塑造。测试通过只意味着”代码做了它做的事”，而非”代码做了它该做的事”。如果你依赖这种验证方式，系统终将漂移——每一次”通过”都在固化一个潜在的错误。

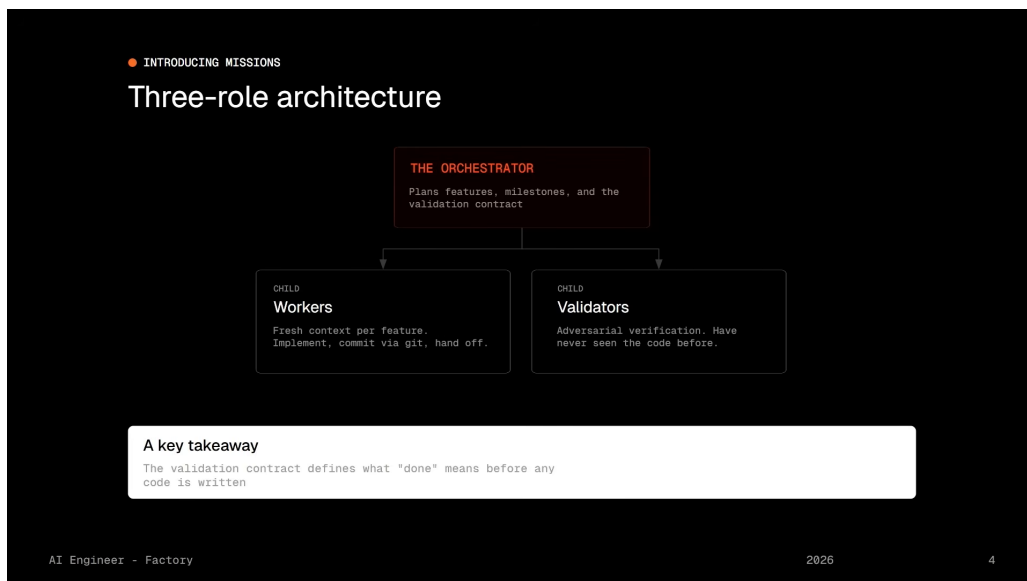


图 5: 验证契约的设计哲学：在编码之前独立定义”完成”标准，避免实现后的确认偏误。图中展示了”先写代码后补测试”与”先定义契约再编码验证”两种范式的根本差异。⁵

3.2.2 验证契约的定义与结构

验证契约 (Validation Contract) 在规划阶段编写，在任意一行代码落地之前就定义了”完成”的标准。它是一个断言集合，独立于实现描述正确性。

对于一个复杂项目，验证契约可能包含数百条断言。每条功能被分配一条或多条断言，该功能必须满足这些断言。所有功能的断言之和必须覆盖契约中的每一条断言——这是一种形式化的覆盖性保证。

验证契约的正式定义

验证契约是在规划阶段由编排器产出的、独立于实现的断言集合，用于在编码开始前定义”完成” (Done) 的客观标准。其核心性质包括：

1. **前置性**：在实现之前编写，避免实现污染预期；
2. **独立性**：不引用任何实现细节，只描述行为预期；
3. **覆盖性**：每个功能至少对应一条断言，所有断言 collectively 覆盖完整契约；
4. **可验证性**：每条断言必须能被审查验证器或用户测试验证器客观检验。

3.2.3 两类验证器：审查验证器与用户测试验证器

在每个里程碑完成后，Missions 运行两类验证器：

1. 审查验证器 (Scrutiny Validator)

这是更”传统”的一类。它运行测试套件、类型检查、lint，并**关键地**——为里程碑内每个已完成的功能生成专门的**代码审查智能体** (Code Review Agent)。这些审查智能体像人类代码审查者一样，逐行检查代码质量、潜在缺陷和架构一致性。

2. 用户测试验证器 (User Testing Validator)

这是更”激进”的一类。它像一个 QA 工程师：启动应用程序，通过**计算机使用能力** (Computer Use) 与之交互——填写表单、检查页面渲染、点击按钮、验证功能流程是否整体贯通。

Luke 透露了一个令人惊讶的数据：Missions 的**挂钟时间** (Wall Clock Time) 大部分都花在这里，等待真实世界的交互执行完成，而非等待生成 token。这说明端到端行为验证是系统可靠性的真正瓶颈，也是真正价值所在。

3.2.4 对抗性验证的设计

两类验证器有一个共同的关键属性：**它们此前从未见过代码**。

这意味着验证器对实现没有投入感，没有”希望代码能工作”的确认偏误。验证天生是**对抗性的** (Adversarial by Design) ——验证器的唯一目标是找出问题，而非确认成功。

⁵ 图 5 截取自演讲视频 00:06:06 处。

常见误解：验证

许多开发者将“验证”等同于“运行测试套件”。Missions 的设计提醒我们：

- 测试覆盖率 100% 不等于功能正确；
- 代码审查者的独立视角和 QA 工程师的端到端交互，是单元测试无法替代的；
- 验证器必须对实现“无知”，才能避免确认偏误。

如果一个系统的验证流程允许实现者同时编写验证逻辑，那么该系统本质上是在自我确认，而非自我检验。

3.3 结构化交接 (Structured Handoff)

验证能够捕获错误，但对于一个运行多天的系统，还有一个同样关键的问题：上下文如何在智能体之间传递而不丢失？

3.3.1 超越“我完成了”

当工作者完成一个功能时，它不会简单地说“我完成了”。相反，它填写一份**结构化交接** (Structured Handoff) 文档，记录以下信息：

- **已完成内容**：具体实现了什么；
- **未完成内容**：哪些部分被搁置或超出范围；
- **运行命令及退出码**：在智能体循环中执行了哪些命令，结果如何；
- **发现的问题**：实现过程中遇到的异常、警告或潜在风险；
- **程序遵守情况**：是否遵循了编排器为该工作者定义的操作规程。

3.3.2 里程碑边界的自我纠正

结构化交接的真正力量在于它与里程碑机制的结合：

1. 错误在**里程碑边界**被捕获——不是即时、零散的，而是在阶段性检查点集中处理；
2. 修正工作被**重新定范围** (Rescoped) ——编排器根据交接内容决定是修复、回滚还是创建后续功能；
3. 任务**自我纠正** (Self-Heals) ——不依赖智能体“记住”发生了什么，而是依赖它们“写下”发生了什么，并据此行动。

Luke 用一句话概括了这一机制的本质：

“Not by hoping that agents remember what happened, but by forcing them to write it down.”

⁶图 6 截取自演讲视频 00:08:09 处。

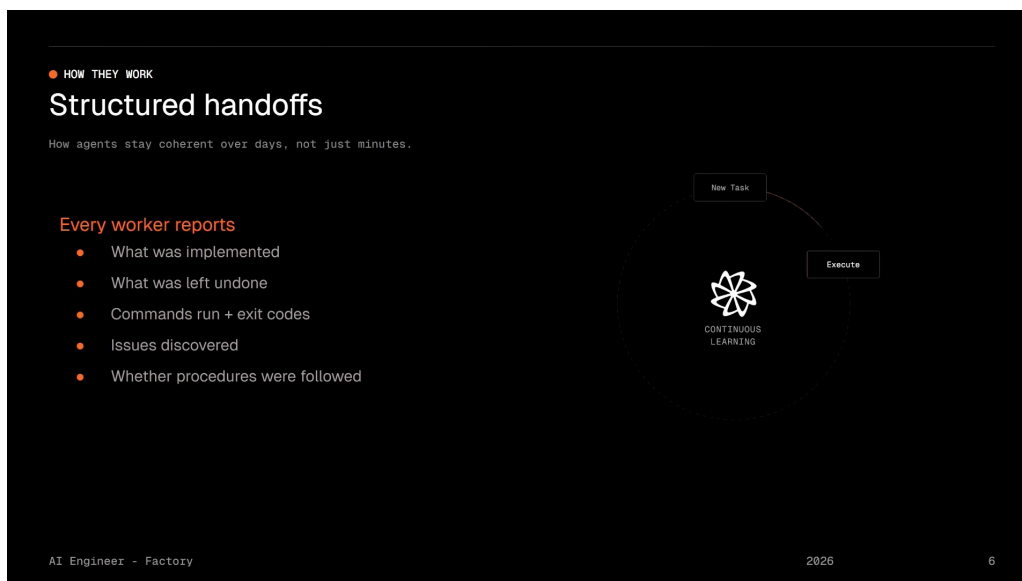


图 6: 结构化交接机制：工作者完成后填写标准化移交文档，错误在里程碑边界被捕获，修正工作被重新定范围，任务自我纠正。⁶

结构化交接的核心机制

结构化交接是工作者完成后填写的标准化状态移交文档，其设计目标是在长期运行中防止上下文衰减和信息丢失。关键机制包括：

- **强制记录**：工作者必须填写预设字段，不能省略；
- **里程碑边界检查**：交接内容在里程碑处被验证器审阅；
- **问题阻断**：未解决的交接问题会阻塞后续进度，直到编排器重新定范围；
- **可追溯性**：每个功能的完整执行历史（命令、退出码、问题）都被持久化。

3.3.3 支撑长期运行的结构

结构化交接与验证契约共同构成了 Missions 能够长期运行的结构性基础。Factory 最长的任务连续运行了 **16 天**——远超过一个完整的开发冲刺（Sprint）。Luke 表示，他们的目标是 **30 天**。

这一目标的可达性，不依赖于某个智能体的记忆力或推理能力的飞跃，而依赖于**系统结构**：验证契约确保每个里程碑的质量基线，结构化交接确保上下文在智能体之间无损传递。两者叠加，使得错误被及时发现、及时修正，系统不会随时间漂移。

3.4 本章小结

本节深入剖析了 Missions 的核心架构。我们了解到：

- Missions 将委托、创造者-验证者、广播和协商四种通信模式组合为统一工作流，排除了缺乏中央协调器的直接通信；
- 三角色架构——编排器（规划与契约）、工作者（干净上下文实现）、验证器（独立视角检验）——分别对应软件工程中的产品/架构、开发和 QA/审查职能；

- **验证契约**在编码前独立定义”完成”标准，从根本上避免了”先写代码后补测试”的确认偏误；
- 两类验证器——审查验证器（测试 + 类型检查 + 代码审查）和用户测试验证器（端到端交互验证）——共同构成对抗性验证体系；
- **结构化交接**通过强制记录和里程碑边界检查，防止长期运行中的上下文丢失，使任务能够自我纠正。

这些设计并非孤立存在，而是相互强化：验证契约为交接提供检验标准，交接为验证提供执行上下文，三角色分工确保每个环节都有专门的智能体负责。正是这种结构性设计，使得 Missions 能够支撑远超人类注意力极限的长期自主运行。

4 执行策略与监控

并行化的诱惑与陷阱

在计算机科学中，“并行化”几乎总是与“加速”同义。当面对 10 个待实现的功能时，直觉反应是启动 10 个智能体同时工作，期望获得 10 倍吞吐。Missions 的设计者最初也遵循了这一直觉——然后他们发现，在软件开发领域，并行化带来的问题往往比收益更多。

4.1 串行执行与内部并行化

4.1.1 为何并行执行在软件开发中失效

Luke 坦诚地分享了 Factory 的实践经验：他们尝试过并行执行，但效果不佳。在软件开发领域，并行智能体会遇到三类冲突：

1. **变更冲突** (Conflicting Changes)：多个智能体同时修改同一代码区域，导致合并冲突甚至逻辑矛盾；
2. **重复工作** (Duplicate Work)：缺乏全局视图时，不同智能体可能独立实现相似功能或重复造轮子；
3. **架构决策不一致** (Inconsistent Architectural Decisions)：每个智能体基于局部最优做出选择，全局架构却因此支离破碎。

这三类问题的共同结果是：**协调开销吞噬了速度收益**，而 token 消耗却在持续燃烧。并行化在纸面上更快，实际上更慢、更贵、更不可靠。

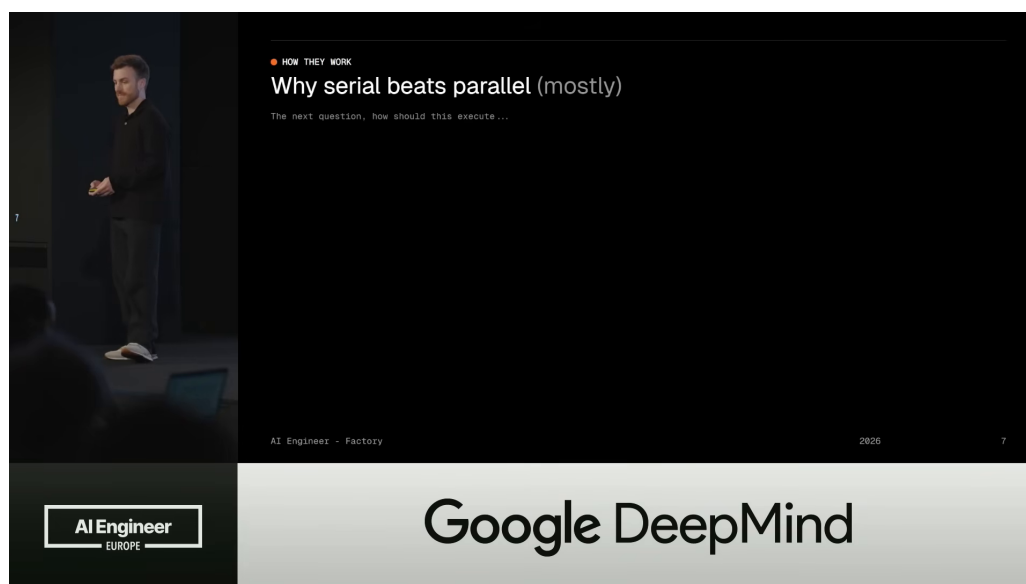


图 7: 串行执行与内部并行化策略：任何时刻只有一个工作者或验证器运行，但在功能内部允许只读操作并行化（代码搜索、API 研究），验证器内部的代码审查也可并行。⁷

⁷图 7 截取自演讲视频 00:09:17 处。

4.1.2 串行执行：全局一致性优先

Missions 的解决方案是**串行执行** (Serial Execution)：任何给定时刻，只有一个工作者或验证器在运行。

这听起来像是一种退步，但 Luke 给出了有力的论证：

”It seems slower on paper, but the error rate drops dramatically, and when you have tasks that run for many days, this sort of correctness compounds.”

对于持续多天的任务而言，**正确性的复利效应**远超短期速度收益。一个每天产生 10% 错误的并行系统，在 16 天后留下的技术债务将是灾难性的；而一个每天产生 1% 错误的串行系统，其累积误差可控得多。

4.1.3 内部并行化：只读操作的安全加速

串行执行并非全盘否定并行化。Missions 在功能**内部**允许**只读操作**的并行化：

- **代码搜索**：在代码库中查找相关函数、类或模式；
- **API 研究**：查询外部文档、接口定义或依赖库的使用方式；
- **代码审查**：验证器为每个功能生成的专门审查智能体可以并行运行。

这些操作的共同特征是**无副作用**——它们读取信息但不修改状态，因此不会产生冲突、重复或不一致。Luke 将这一策略概括为：

串行执行与内部并行化的定义

Missions 的执行策略是一种**串行执行与内部并行化** (Serial Execution with Targeted Internal Parallelization) 的混合模式：

- **全局串行**：任何时刻只有一个工作者或验证器运行，确保代码库状态的一致性；
- **内部并行**：在单个功能或验证器内部，只读操作（搜索、研究、审查）可以并行执行；
- **无副作用约束**：并行化仅限于不修改共享状态的操作，从根本上消除冲突可能。

这一策略的深层原理是：在软件开发中，**写操作的协调成本远高于读操作的并行收益**。Missions 选择保守地串行化写操作，同时最大化读操作的并行度。

4.2 Mission Control：监控与异步运行

当任务可能持续数天甚至数周时，标准的聊天界面显然不再适用。用户需要一种方式，在任意时刻快速了解“项目进展到哪了”“预算还剩多少”“当前谁在做什么”。

4.2.1 超越聊天界面的监控需求

Luke 指出了标准聊天界面的根本局限：

- 无法一眼看出项目完成的百分比；
- 无法追踪预算消耗与初始设定的对比；

- 无法了解当前活跃工作者的具体行为和上下文；
- 无法查看验证器发现的问题及系统的后续调整计划。

对于持续几分钟的会话，这些信息的缺失尚可忍受；对于持续数天的任务，这种信息黑洞是不可接受的。

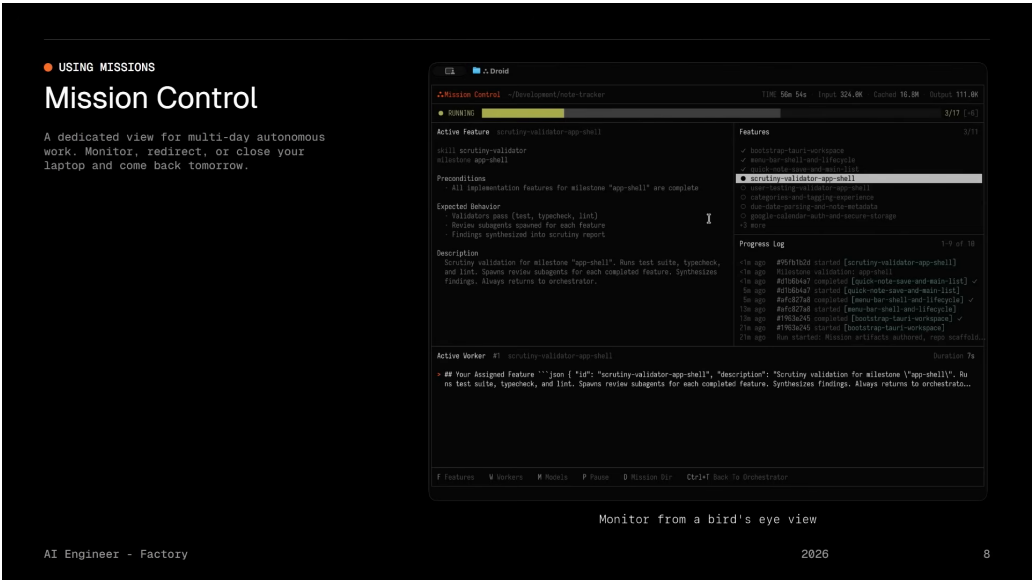


图 8: Mission Control 专用视图：展示项目完成进度、预算消耗、当前活跃工作者、交接摘要、验证器发现的问题及后续调整计划。⁸

4.2.2 Mission Control 的核心功能

Factory 为此构建了 **Mission Control**（任务控制中心）——一个专为长期任务设计的监控视图。它提供以下关键信息：

- **项目进度**：已完成的功能、当前的里程碑、剩余工作量；
- **预算追踪**：已消耗的 token 和费用相对于初始预算的比例；
- **活跃工作者状态**：当前正在执行的工作者、其正在处理的功能、已运行时间；
- **交接摘要**：最近完成的工作者填写的结构化交接要点；
- **验证器发现**：审查验证器和用户测试验证器报告的问题；
- **后续调整**：编排器基于验证结果对计划的修正（重新定范围、创建后续功能等）。

4.2.3 两种监督模式

Mission Control 支持两种截然不同的使用模式：

1. 项目经理模式

⁸图 8 截取自演讲视频 00:10:30 处。

用户作为项目经理 (Project Manager) 主动监督实施过程。定期查看 Mission Control, 审查交接摘要, 在关键决策点介入——例如批准编排器提出的计划调整, 或在验证器发现严重问题时决定修复策略。

2. 完全异步模式

用户设置好任务后”去和朋友出去玩”——Luke 用了这个生动的比喻。Mission Control 让任务可以完全异步运行, 用户只需在方便时查看状态, 无需持续在线监督。这是 Missions 真正解放人类注意力的体现: 不是把人类从决策中移除, 而是把人类从持续监督中解放出来。

隐藏假设: 用户仍保有最终决策权

Mission Control 的设计隐含一个重要假设: 用户仍然是最终决策者, 而非被完全排除在外。编排器会提出计划调整和范围变更, 但这些通常需要用户批准。完全异步运行是可能的, 但在关键节点 (如预算超限、验证反复失败) 系统仍会等待人类输入。这意味着 Missions 不是”取代人类”, 而是”放大人类注意力”——让一个人能够同时监督更多工作流, 而非让一个人完全退出工作流。

4.3 本章小结

本节探讨了 Missions 的执行策略和监控机制:

- **并行执行**在软件开发领域效果不佳, 因为智能体冲突、重复工作和架构决策不一致会吞噬速度收益;
- Missions 采用**串行执行**, 任何时刻只有一个工作者或验证器运行, 以全局一致性换取长期正确性的复利;
- 在功能**内部**允许**只读操作**的并行化 (代码搜索、API 研究、代码审查), 在无副作用的前提下最大化效率;
- **Mission Control** 提供专用视图, 展示进度、预算、活跃工作者、交接摘要、验证发现和后续调整;
- 支持**项目经理监督**和**完全异步运行**两种模式, 真正解放人类注意力。

执行策略的选择体现了 Missions 的一个核心设计哲学: 在”更快”和”更正确”之间, 优先选择”更正确”。对于运行数天的任务而言, 一次错误的代价远高于一次等待的收益。Mission Control 则确保, 即使在完全异步模式下, 人类也能在需要时迅速掌握全局、做出决策。

5 模型选择、生产数据与架构未来

前置知识回顾

在深入本章之前，建议读者已理解 Missions 的三角角色架构 (3)：编排器 (Orchestrator) 负责规划与产出验证契约，工作者 (Workers) 负责实现，验证器 (Validators) 负责对抗性验证。本章将在此基础上回答一个关键问题：当系统可以运行数天时，如何为不同角色选择最合适的模型，以及生产数据如何验证这套架构的有效性。

5.1 按角色选择模型：Droid Whispering

Missions 的三角角色架构隐含了一个常被忽视的前提：没有单一模型或供应商能在规划、实现、验证三者上都做到最强。演讲者 Luke Alvoeiro 指出，这是使用 Missions 这类系统时必须面对的现实——不同角色对模型的能力需求截然不同。

三角角色的模型能力需求矩阵

角色	核心能力需求	推理特征
编排器 (Orchestrator)	缓慢、仔细的推理	深度规划、需求澄清、架构决策
工作者 (Workers)	快速编码流畅度与创造力	代码生成、问题解决、API 研究
验证器 (Validators)	精确遵循指令	测试执行、类型检查、对抗性审查

规划需要模型具备慢思考 (slow reasoning) 能力——能够梳理复杂需求、识别隐含假设、产出可验证的计划。实现则需要快速编码流畅度 (fast code fluency)——模型需要熟悉多种编程语言、框架和工具链，能够高效地将规格说明转化为可运行代码。验证则要求精确遵循指令 (precise instruction following)——验证器必须严格按照验证契约执行检查，不能自行放宽标准或忽略边界条件。

常见误解：最强模型

许多团队倾向于为所有角色使用同一个“最强”模型 (如当时最新的 GPT-4 或 Claude 系列)，认为这样可以简化运维。然而，模型的“强”是维度性的：一个在代码生成上表现优异的模型，可能在严格遵循复杂验证指令时出错；一个推理能力突出的模型，编码速度却可能不够理想。更重要的是，锁定单一供应商意味着被该模型家族的最弱能力所约束——“你只有最弱的一环那么强” (You're only as strong as your weakest link)。

面对这种需求分化，Factory 内部发展出了一种被称为 Droid Whispering (机器人耳语) 的新技能。它并非某种具体技术，而是一种系统性的直觉：

- 心理建模**：在心理层面建模不同大语言模型 (LLM) 的交互方式——它们如何响应不同类型的提示、在何种任务上容易出错、错误模式是什么。
- 失败累积分析**：理解单个模型的失败如何在多日运行中累积。一个工作者在实现阶段的微小偏差，如果未被验证器捕获，可能在后续里程碑中被放大为系统性错误。
- 有意识的座位分配**：基于上述分析，为每个角色 (“座位”) 选择最合适的模型。例如，验证器甚至可以使用与工作者完全不同的供应商模型，以避免训练数据偏误 (training data bias) 导致的“同温层”效应。

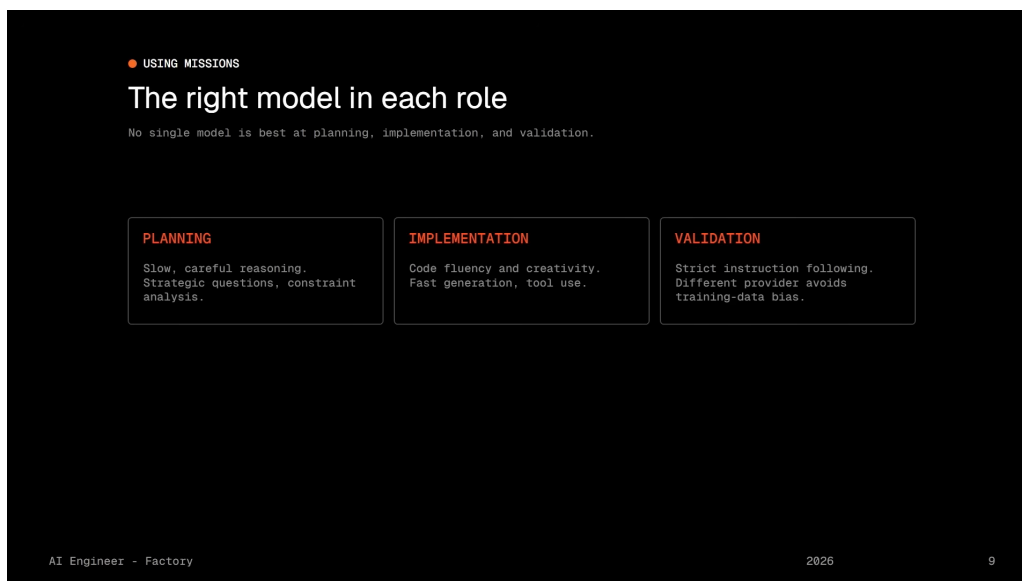


图 9: Droid Whispering: 按角色需求为编排器、工作者、验证器分配合适模型的策略示意。不同模型供应商的强项各异，模型无关架构（Model-Agnostic Architecture）允许系统利用各供应商的最优能力。⁹

演讲者特别提到，构建 Missions 原型的工程师 Theo 制定了默认模型配置，但 Factory 鼓励用户根据项目需求自行调整。这种灵活性带来了双向优势：一方面，正确的模型选择能产生**复合优势**（compounding advantage）——随着模型持续专业化，为每个座位匹配最优模型的收益会随时间放大；另一方面，Missions 的结构化机制（验证契约、里程碑检查点）也能**补偿非前沿模型的不足**，使得开源权重模型（Open-Weight Models）也能成功运行复杂任务。

5.2 生产数据分析：构建 Slack 克隆

理论架构的有效性最终需要在生产环境中得到验证。演讲者以**构建 Slack 克隆**这一实际任务为例，展示了 Missions 在生产中的真实数据表现。

Slack 克隆任务关键数据指标

- **实现占比**：60% 的时间和 token 消耗在实现阶段，说明即便在高度自动化的系统中，编码仍是主要工作量。
- **验证从不一次通过**：验证几乎从不在第一次尝试时成功，这直接证明了 QA 循环（QA loop）的价值——如果系统没有对抗性验证机制，这些缺陷将被遗漏。
- **后续功能需求**：任务过程中几乎总是需要创建后续功能（follow-up features），说明复杂软件的开发天然具有迭代性，自动化系统必须支持这种迭代。
- **测试密度**：最终代码中 50% 的行数是测试代码，90% 的代码被测试覆盖。
- **提示缓存**：大量使用提示缓存（Prompt Caching）来抵消长任务运行的成本。

⁹视频画面时间区间：00:11:22–00:13:03。

¹⁰视频画面时间区间：00:13:03–00:14:33。



图 10: Slack 克隆任务的生产数据分析。图中展示了时间/token 分布、验证循环、测试覆盖率及提示缓存的使用情况。¹⁰

这组数据揭示了几个深层洞察。首先，**验证不是可选项而是必需品**——如果验证一次通过，反而说明验证标准可能过于宽松。其次，**测试代码的“膨胀”是健康信号**：在 Missions 的驱动下，系统主动生成了大量端到端测试和单元测试，这些测试不仅验证当前功能，更为后续迭代提供了安全网。最后，**提示缓存的经济性不可忽视**——当任务运行数天时，上下文窗口的重用成为控制成本的关键手段。

提示缓存（Prompt Caching）的经济学

提示缓存是一种重用大段提示上下文的技术。在长任务中，系统提示（system prompt）、代码库结构、已完成的规格说明等大量文本在不同步骤间重复出现。通过缓存这些上下文，可以显著降低重复 token 的计费成本。对于运行数天的 Missions 任务，提示缓存从“优化手段”变为“经济必需”。

5.3 企业应用与 Bitter Lesson

Missions 在企业场景中已展现出广泛适用性。演讲者列举了 Factory 客户的主要使用场景：

- **快速原型**：overnight 内将新想法转化为可演示原型
- **内部工具**：以越来越快的速度构建内部工具和自动化脚本
- **大规模重构与迁移**：执行跨越整个代码库的重构和框架迁移
- **ML 研究**：支持机器学习实验的代码生成与数据处理流程
- **代码库现代化**：改造遗留代码库，使其对智能体更友好、对人类更易维护

¹¹ 视频画面时间区间：00:14:33–00:15:51。

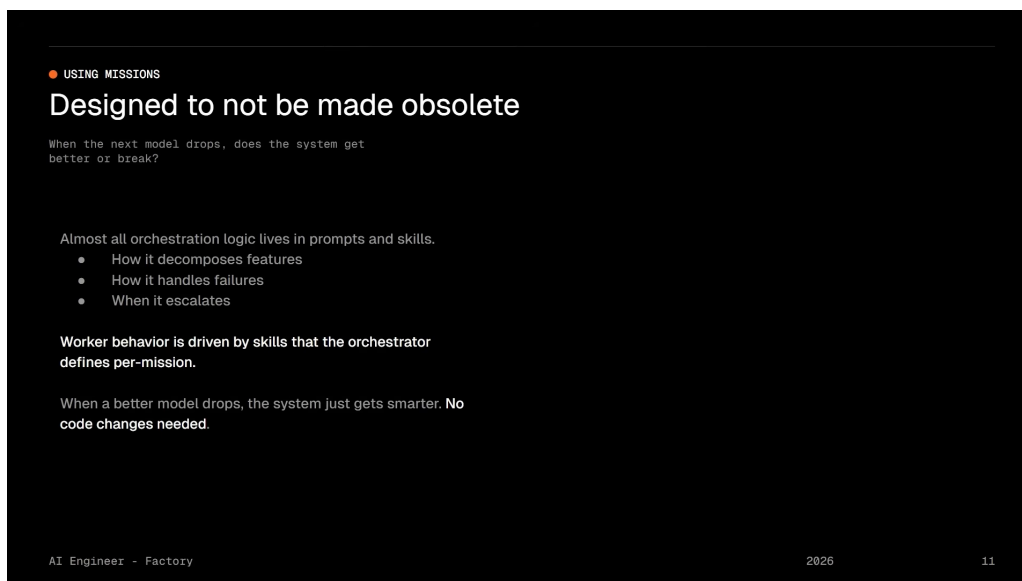


图 11: 企业应用场景与 Bitter Lesson 的应对策略。Missions 的编排逻辑几乎完全定义在提示和技能中，仅保留极薄的确定性逻辑用于记账。¹¹

然而，在讨论企业应用时，演讲者引入了一个深刻概念——**Bitter Lesson（苦涩教训）**。这是每一个构建多智能体系统的人都面临的恐惧：下一版模型发布可能让现有架构一夜过时。当 GPT-5、Claude 4 或 Gemini 3 出现时，今天精心设计的编排逻辑是否还有价值？

Missions 的设计哲学正是对这一恐惧的直接回应：

Missions 的”抗过时”设计原则

1. **编排逻辑提示化**：几乎全部的编排逻辑定义在提示（prompts）和技能（skills）中，而非硬编码的状态机。约 700 行文本即可定义特征分解、失败处理等核心策略，修改其中四句话就能显著改变执行策略。
2. **工作者行为技能化**：每个任务的工作者行为由编排器为该任务定义的技能驱动，因此可以获得高度定制化的行为。
3. **确定性逻辑最小化**：系统中唯一的确定性逻辑非常薄，仅聚焦于记账——运行验证、在交接问题未解决时阻塞进度。这部分逻辑不替代模型的决策能力，而是**让模型做它们最擅长的事**。
4. **使用熟悉的原语**：系统使用智能体已经熟悉的原语（如 `agents.md`、技能文件等），使模型能够直接理解并执行。

这种设计的本质是**让系统随模型改进而提升**。当模型变得更擅长规划时，编排器的提示无需大幅修改即可获得更好的计划；当模型编码能力增强时，工作者的实现质量自然提升；当模型验证更精确时，验证器的检出率随之提高。Missions 提供的是**纪律（discipline）**——确保模型在正确的框架内工作；模型提供的是**智能（intelligence）**——在框架内做出高质量决策。

5.4 本章小结

本章从三个维度完成了对 Missions 系统的补充论证：

1. **模型选择维度**：Droid Whispering 揭示了多智能体系统的隐性成本——不是每个模型都适合每个角色，正确的”座位分配”能产生复合优势。
2. **生产验证维度**：Slack 克隆的数据证明，即使在自动化程度极高的系统中，验证循环、测试生成和提示缓存仍是不可或缺的工程实践。
3. **架构韧性维度**：通过将编排逻辑提示化、将确定性逻辑最小化，Missions 构建了一个随模型代际提升而自然进化的系统，有效规避了 Bitter Lesson 的风险。

这三个维度共同指向一个核心结论：**多智能体系统的竞争力不仅来自单个组件的设计，更来自组件之间的匹配与系统的整体演化策略。**

6 总结与延伸

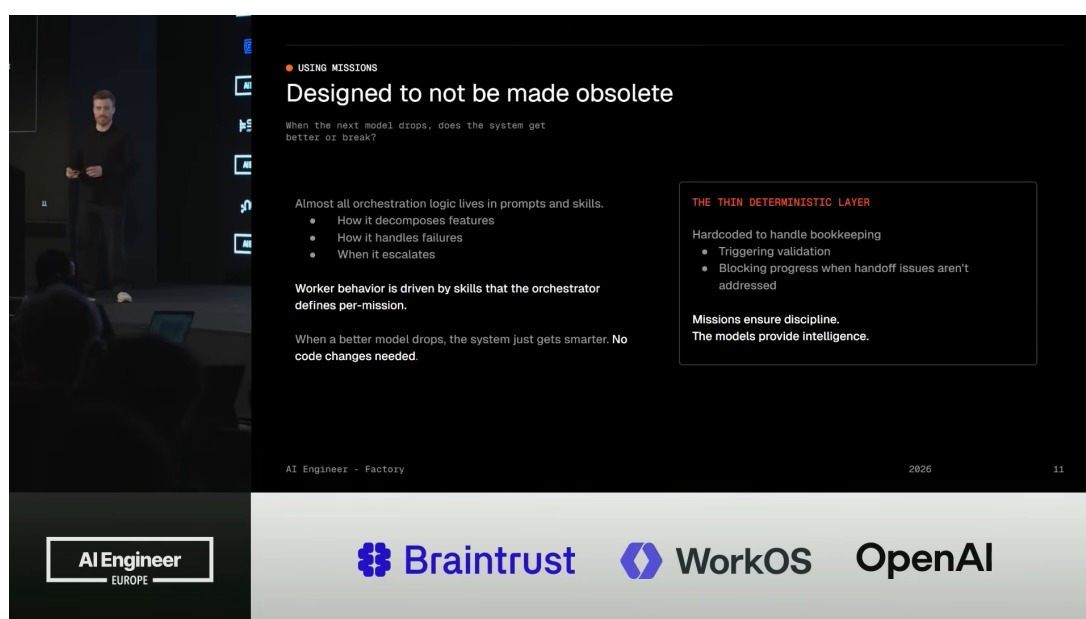


图 12: 软件工程经济学的转变：从”人类注意力是瓶颈”到”结构化智能体系统释放注意力”。¹²

6.1 演讲者的核心主张回顾

Luke Alvoeiro 在演讲的实质性结尾部分（排除最后的日常寒暄与行动号召）中，将全篇论点凝聚为几个关键主张：

¹²视频画面时间区间：00:15:51–00:18:14。

演讲者的核心主张

1. **瓶颈转移**：软件工程的核心瓶颈已从“智能不足”转变为“人类注意力有限”。模型足够聪明，但缺乏监督带宽。
2. **经济学转变**：一个 5 人工程师团队可能从同时推进 10 个工作流提升到 30 个。团队可专注于架构和产品决策，而非执行本身。
3. **代码库净收益**：任务完成后，代码库比开始时更干净——端到端测试、单元测试、技能和结构使人类和智能体在未来的工作中都更高效。
4. **策略组合**：Missions 是五种原始策略的组合——委托（Delegation）、创造者-验证者（Creator-Verifier）、广播（Broadcast）、协商（Negotiation）。
5. **连接组织的重要性**：仅有策略不够，还需要结构化交接（Structured Handoff）、正确的模型选择（Droid Whispering）、以及随模型改进的架构。
6. **未来属于生态思维者**：理解智能体生态系统（Agent Ecosystem）和模型在压力下如何组合的人，将推动下一代创新。

6.2 核心主张的结构化提炼

基于以上主张，我们可以将 Missions 的核心理念提炼为一个三层模型：

第一层：问题诊断——人类注意力是稀缺资源。工程师有 50 个功能待办，但每天只能推进几个，因为每个任务都需要关注和审查。这不是因为模型不够聪明，而是因为人类无法同时监督 50 个并行的智能体会话。

第二层：解决方案架构——Missions 通过三种机制解决注意力瓶颈：

- **结构化分工**：三角色架构（编排器-工作者-验证器）将人类从执行细节中解放出来，只需在关键决策点（需求澄清、计划审批、里程碑审查）介入。
- **对抗性验证**：验证契约在编码前定义“完成”标准，验证器从未见过实现代码，确保验证的天然对抗性。错误在里程碑边界被捕获并修正。
- **异步监督**：Mission Control 支持完全异步运行，用户可以像项目经理一样监督，也可以完全放手。

第三层：演化策略——系统通过提示驱动设计和模型无关架构，确保自身随底层模型能力提升而自然进化，而非被其淘汰。

6.3 跨章节综合与概念压缩

将全篇五个章节串联，可以识别出一条清晰的认知递进路径：

章节	核心问题	回答
1	为什么需要多智能体？	人类注意力是瓶颈
2	有哪些通信模式可用？	五种基本框架
3	如何组合这些模式？	三角色 + 验证契约 + 结构化交接
4	如何执行和监控？	串行执行 + Mission Control
5	如何让系统持久有效？	Droid Whispering + 提示驱动架构

这条路径揭示了一个深层模式：**Missions 的每一层设计都在处理“规模”带来的挑战**。五种框架分类处理的是**通信规模**——多个智能体如何有效交换信息；三角色架构处理的是**分工规模**——如何将复杂任务分解为可管理的子任务；验证契约处理的是**质量保证规模**——如何在长时间运行中防止漂移；串行执行处理的是**协调规模**——如何在避免冲突的同时利用并行性；Droid Whispering 处理的是**能力规模**——如何在模型能力不断演进时保持系统最优。

Missions 与经典软件工程的类比

Missions 的设计哲学与经典软件工程中的多个原则形成有趣的对照：

- **验证契约** ↔ **测试驱动开发 (TDD)**：两者都强调在实现前定义正确性标准，避免确认偏误。
- **结构化交接** ↔ **文档化与知识管理**：两者都解决“信息随人员流动而丢失”的问题。
- **串行执行 + 内部并行** ↔ **乐观并发控制**：两者都在保证一致性的前提下最大化并行度。
- **提示驱动架构** ↔ **配置优于约定**：两者都追求灵活性，将变化频繁的部分提取到易修改的层面。

6.4 具体收获

对于希望在自己的项目中应用这些理念的读者，以下是可直接落地的收获：

1. **从单智能体到多智能体的第一步**：不要急于构建复杂的并行系统。先识别你的任务中天然存在的“委托”和“创造者-验证者”模式，用这两个最基础的模式搭建第一个多智能体工作流。
2. **验证先于实现**：无论使用何种框架，在编码前定义“完成”标准。这可以是一个简单的检查清单，也可以是一组自动化测试。关键是让验证者（无论是人还是智能体）在验证时不受实现细节的影响。
3. **为不同任务选择不同模型**：即使你只有一个 API 密钥，也可以利用同一供应商的不同模型版本。将需要深度推理的任务分配给推理型模型，将需要快速编码的任务分配给代码型模型。
4. **设计“可进化”的系统**：将业务逻辑放在提示和配置中，将确定性逻辑限制在记账和流程控制。这样当新模型发布时，你只需调整提示即可获得提升，而非重写核心代码。
5. **接受验证的“不完美”**：Slack 克隆的数据告诉我们，验证从不一次通过是正常的。一个健康的系统应该有预期地处理验证失败，将其视为改进机会而非系统缺陷。

6.5 开放问题与后续步骤

演讲者在结尾明确提出了两个开放问题，它们也构成了本课程笔记的延伸方向：

开放问题：Missions 的未解挑战

1. **进一步并行化**：如何在保持正确性的前提下进一步并行化 Missions 的工作负载，使其运行更快？当前串行执行 + 只读并行化的策略是保守的，但软件开发中的写冲突问题本质上是困难的。未来的突破可能来自更智能的冲突预测、代码区域的自动划分、或基于语义而非文本的合并策略。
2. **Missions 的编排**：如何开始将 Missions 本身编排进更复杂的工作流？这意味着单个 Mission 成为更大系统的原子单元，多个 Mission 之间需要协调依赖、共享资源和同步状态。这本质上是从“单 Mission 管理”到“Mission 集群管理”的跃迁。

此外，基于全篇内容，我们还可以提出以下延伸思考：

- **验证契约的形式化**：当前验证契约以自然语言断言为主，能否发展出更形式化的规格说明语言，使验证契约既可被人类理解、又可被机器精确执行？
- **结构化交接的标准化**：Missions 的交接格式是专有的，行业内能否形成跨平台的标准化交接协议，使不同厂商的智能体能够无缝协作？
- **Droid Whispering 的自动化**：模型选择目前依赖人类直觉，能否构建元模型（meta-model）来自动评估不同模型在特定任务上的表现，并动态调整座位分配？
- **成本-质量权衡的量化**：当任务运行数天时，token 成本、时间成本和输出质量之间的帕累托前沿是什么？如何为不同场景找到最优配置？

6.6 结语

Luke Alvoeiro 的演讲以一句行动号召结束：“Open Droid，尝试运行 /missions，与编排器争论范围，批准计划，然后去做别的事情。”这句话浓缩了 Missions 的全部承诺——**让软件工程从“人类必须全程在场”转变为“人类只需在关键节点决策”**。

从更宏观的视角看，Missions 代表了一种新的软件工程范式：不是用智能体替代工程师，而是**用结构化系统扩展工程师的注意力边界**。在这个范式中，工程师的角色从“亲自编写每一行代码”演变为“设计智能体协作的架构、定义正确性标准、在关键决策点把关”。代码库因此变得更干净、更可维护、更易于人类和智能体共同工作。

未来属于那些能够理解智能体生态系统、能够在模型能力的快速演进中保持系统韧性、能够将多智能体通信的抽象模式转化为实际可运行系统的人。Missions 提供了一套经过生产验证的蓝图，但真正的创新将来自每一位实践者在自己的领域中对这些模式的重新组合与演化。